

2018-06-30 TRAFFIC ANALYSIS EXERCISE: ANSWERS

SORTING THROUGH THE ALERTS

www.malware-traffic-analysis.net/2018/06/30/2018-06-30-traffic-analysis-exercise.pcap.zip

www.malware-traffic-analysis.net/2018/06/30/2018-06-30-traffic-analysis-exercise-alerts.zip

Zip files are password-protected with the standard password. See the "about" page of my site if you don't know it.

The associated network segment:

- LAN segment: 172.16.2.0/24 (172.16.2.0 through 172.16.2.255)
- Broadcast address: 172.16.2.255
- Domain controller: 172.16.2.9 (FabulousDuke-DC)
- Domain: fabulousdukes.com

REPORT GUIDELINES:

EXECUTIVE SUMMARY:

- Date/time of the activity in UTC
- What happened
- Who it happened to

DETAILS:

- The internal IP address you're investigating: 172.16.2.169
- The MAC address for 172.16.2.169
- The Host name for 172.16.2.169
- The Windows account name, if 172.16.2.169 is a Windows host

INDICATORS:

List the IP addresses, ports, and domains directly related to any malicious activity you can verify from the pcap.

ANSWERS

EXECUTIVE SUMMARY:

On Saturday 2018-06-30 at 20:27 UTC, we received alerts indicating a Windows computer used by Lloyd Maxwell downloaded malware disguised as an update to Chrome from a fake Google Chrome update page. Further investigation revealed the user was possibly infected with Dridex due to this activity.

DETAILS:

2018-06-30 TRAFFIC ANALYSIS EXERCISE: ANSWERS

- IP address: 172.16.2.169
- MAC address: 00:13:92:06:5c:7b
- Host name: Conservator-PC
- User account name: lloyd.maxwell

INDICATORS:

- 93.95.100.178 port 80 - **lw2e.sineadhollywoodnutt.com** - various HTTP requests for fake Chrome update page
- 93.95.100.138 port 80 - **bb17eb22.login3.kimbrelelectric.com** - POST /blank.gif
- 107.170.231.118 port 4143 - IP and TCP port associated with Dridex (Feodo) - attempted TCP connections, but no response from the server

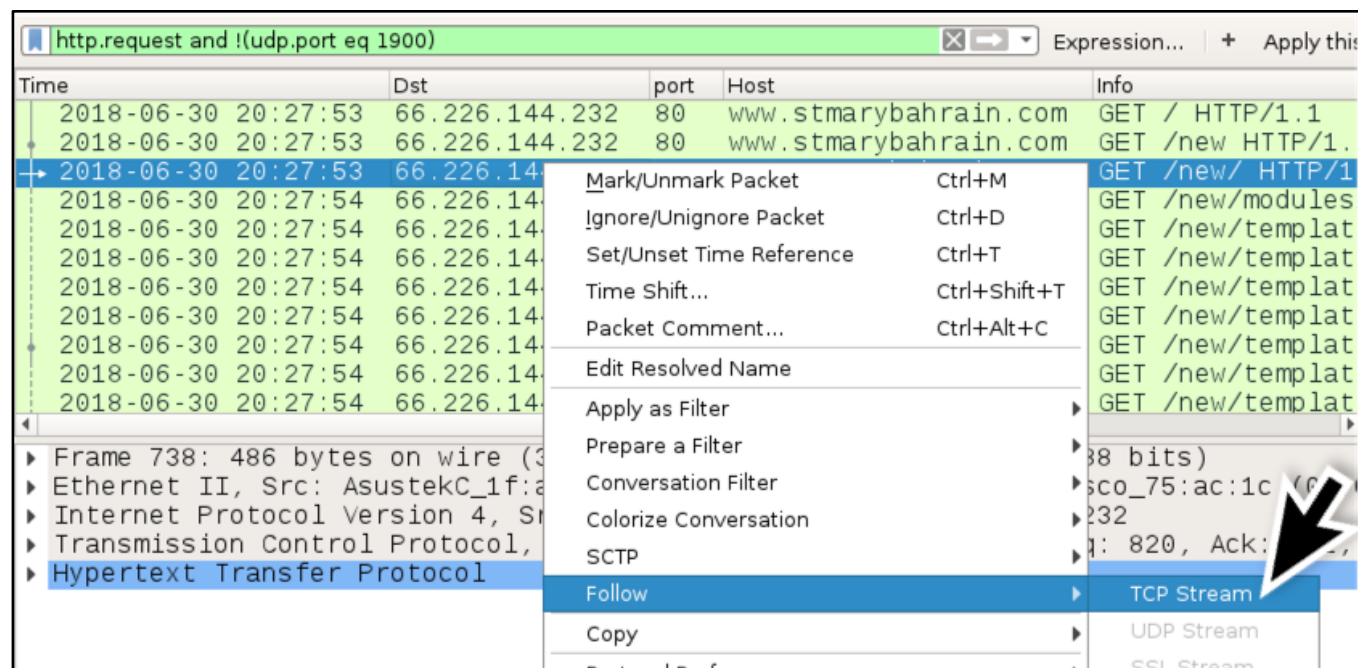
DETAILS

As always, I recommend you set up Wireshark according to the tutorials I've documented at:

- <http://malware-traffic-analysis.net/tutorials/index.html>

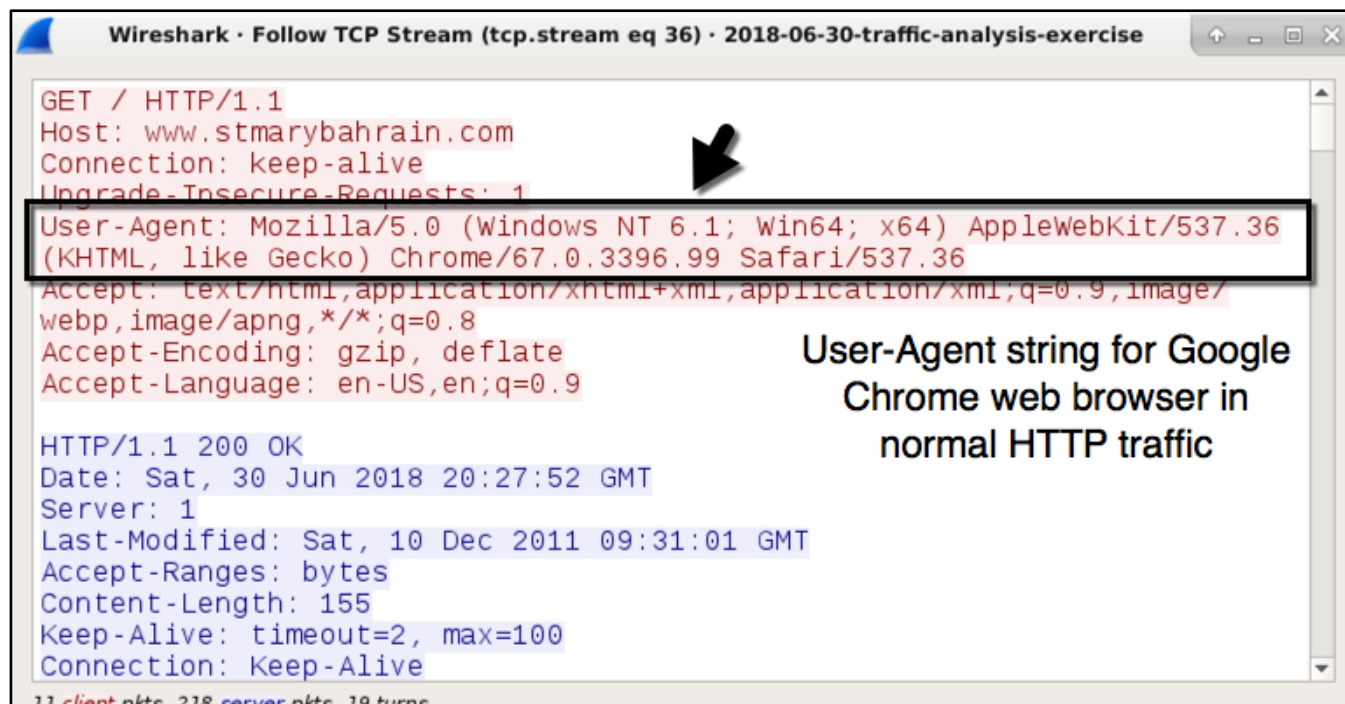
First, is this a Windows host? It sure is!

Filter on **http.request and !(udp.port eq 1900)** to see some of the HTTP GET requests at the beginning of this pcap. Follow any of the first few HTTP GET requests to **www.stmarybahrain.com**, and you'll see **Windows NT 6.1; Win64; x64** listed in the User-Agent string. Windows NT 6.1 represents Windows 7. This is also Google Chrome being used as the web browser.



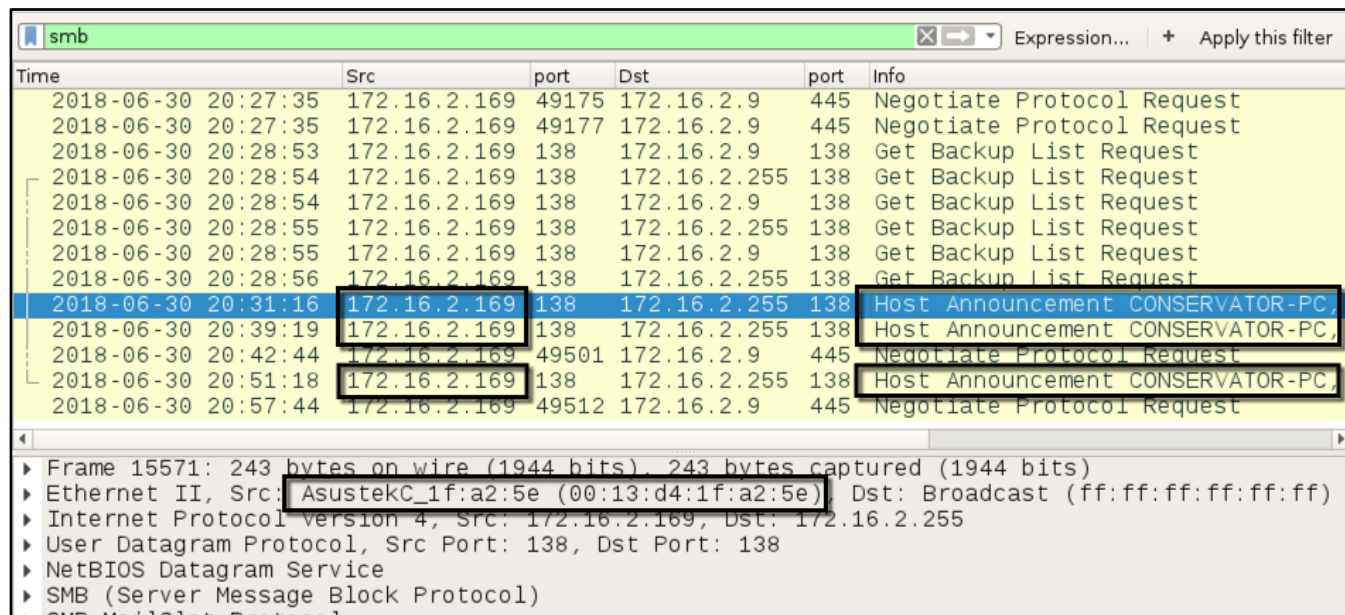
Shown above: Filtering on **http.request and !(udp.port eq 1900)** and following one of the TCP streams.

2018-06-30 TRAFFIC ANALYSIS EXERCISE: ANSWERS



Shown above: User-Agent string in the HTTP header for Google Chrome on a Windows 7 x64 host.

Who is this? For this month's exercise, we don't have any DHCP or NBNS traffic for the host name. However, SMB traffic reveals the host name of 172.16.2.169 through host announcements. Filter on **smb** and you should see the following information in your column list (note CONSERVATOR-PC in the host announcements):

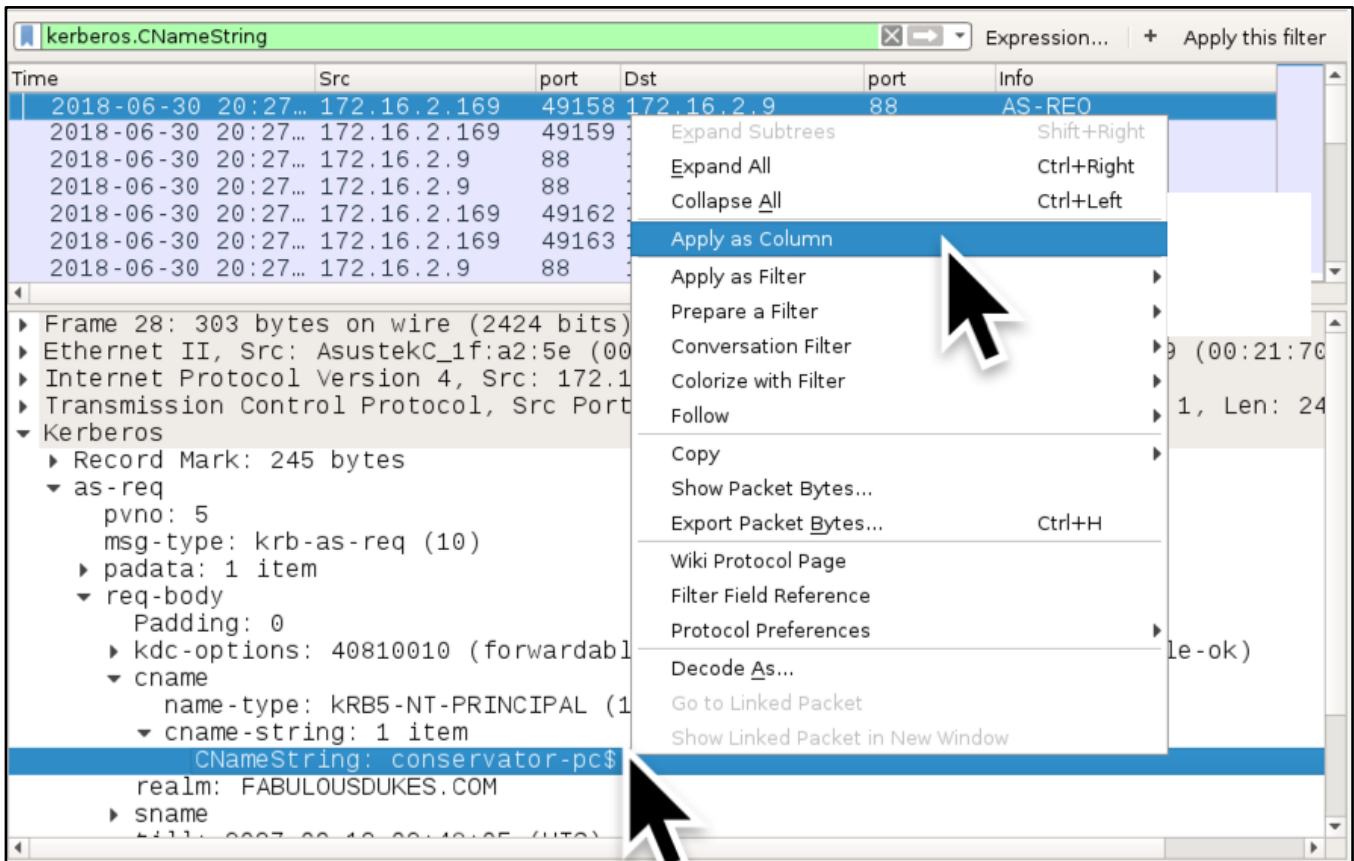


Shown above: Correlating the IP address, host name, and MAC address from SMB traffic.

In this pcap, Kerberos traffic will get us the Windows account name. Filter on **kerberos.CNameString** and go to the frame details section. Select the first frame.

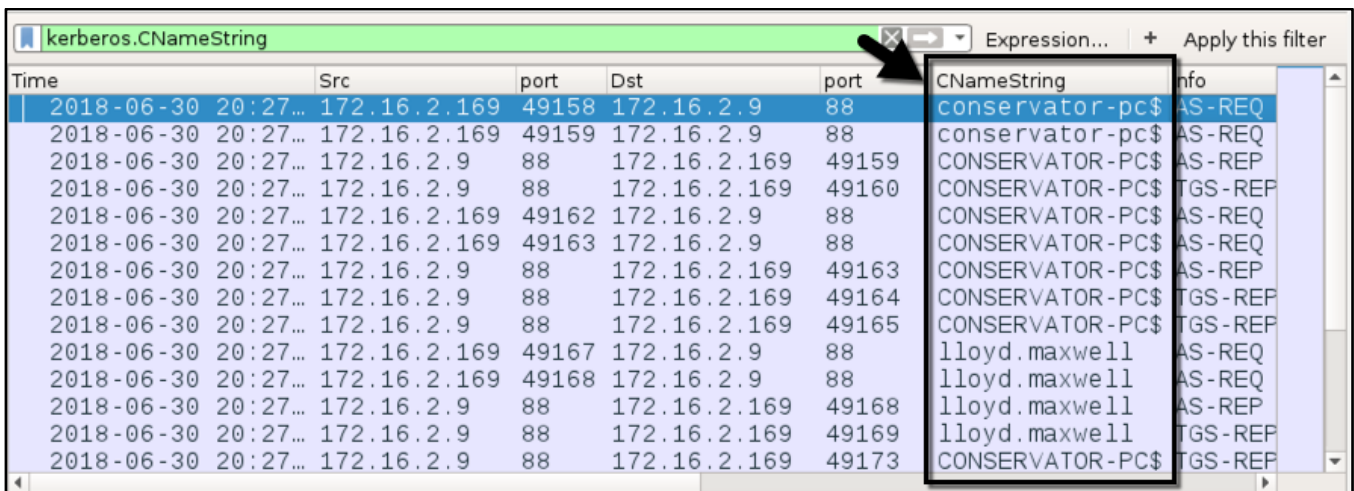
2018-06-30 TRAFFIC ANALYSIS EXERCISE: ANSWERS

In the same details, expand Kerberos as shown below. **Kerberos** → **as-req** → **req-body** → **cname** → **cname-string** → **CNameString**. Left-click on **CNameString** to select it, then right-click to bring up a menu and select "Apply as Column" for a new column.



Shown above: Filter on **Kerberos.CNameString**, finding the **CNameString** value, and applying as a column.

You should now have a new column named "CNameString" in your column display. In the column display, any items that end with a \$ are host names. Any that don't end with a \$ are Windows account names. This should reveal **CONSERVATOR-PC** as the host name for 172.16.2.169 and **lloyd.maxwell** as the Windows account name.



Shown above: The **CNameString** values.

2018-06-30 TRAFFIC ANALYSIS EXERCISE: ANSWERS

What time did this activity happen? Traffic from the pcap starts on Saturday 2018-06-30 at 20:27 UTC. If you correlate the IP addresses/ports from the alerts with the appropriate IP addresses/ports from the pcap, you'll find the alerts also start at 20:27 UTC. This is also shown in the alerts list from the alerts archive for this exercise.

Now we have the everything needed for the executive summary, except for what actually happened. Let's look at the alerts.

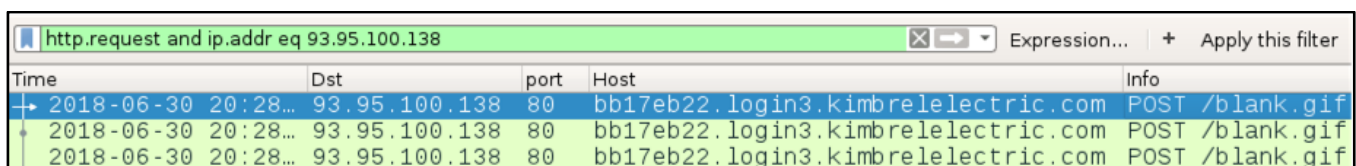
Src IP	SPort	Dst IP	DPort	Pr	Event Message
66.226.144.232	80	172.16.2.169	49199	6	GPL WEB_CLIENT web bug 0x0 gif attempt
52.207.135.168	443	172.16.2.169	49422	6	ET POLICY Lets Encrypt Free SSL Cert Observed
52.10.154.150	443	172.16.2.169	49421	6	ET POLICY Lets Encrypt Free SSL Cert Observed
172.16.2.169	49440	93.95.100.138	80	6	ET POLICY Data POST to an image file (gif)
172.16.2.169	49440	93.95.100.138	80	6	ETPRO TROJAN POST to a gif file
172.16.2.169	49440	93.95.100.138	80	6	ETPRO CURRENT_EVENTS JS.SocGholish POST Request
172.16.2.9	53	172.16.2.169	64260	17	GPL DNS SPOOF query response with TTL of 1 min. and ...
172.16.2.169	49471	107.170.231.118	4143	6	ET CNC Feodo Tracker Reported CnC Server group 2
173.194.184.12	80	172.16.2.169	49479	6	ET POLICY ZIPPED EXE in transit
173.194.184.12	80	172.16.2.169	49482	6	ET POLICY ZIP file download
172.16.2.9	53	172.16.2.169	54161	17	ET DNS Standard query response, Name Error

Shown above: Alerts for this exercise.

In the alert list, let's ignore all of the POLICY alerts, the GPL WEB_CLIENT alert, and the DNS alerts. Instead, let's focus on the CNC, CURRENT_EVENTS and TROJAN alerts. Our focused list is:

- 172.16.2.169 49440 - 93.95.100.138 port 80 - **ETPRO TROJAN POST to a gif file**
- 172.16.2.169 49440 - 93.95.100.138 port 80 - **ETPRO CURRENT_EVENTS JS.SocGholish POST Request**
- 172.16.2.169 49471 - 107.170.231.118 port 80 - **ET CNC Feodo Tracker Reported CnC Server group 2**

Let's look at the HTTP requests for the first two alerts on 93.96.100.138 over TCP port 80 by filtering on **http.request and ip.addr eq 93.96.100.138**

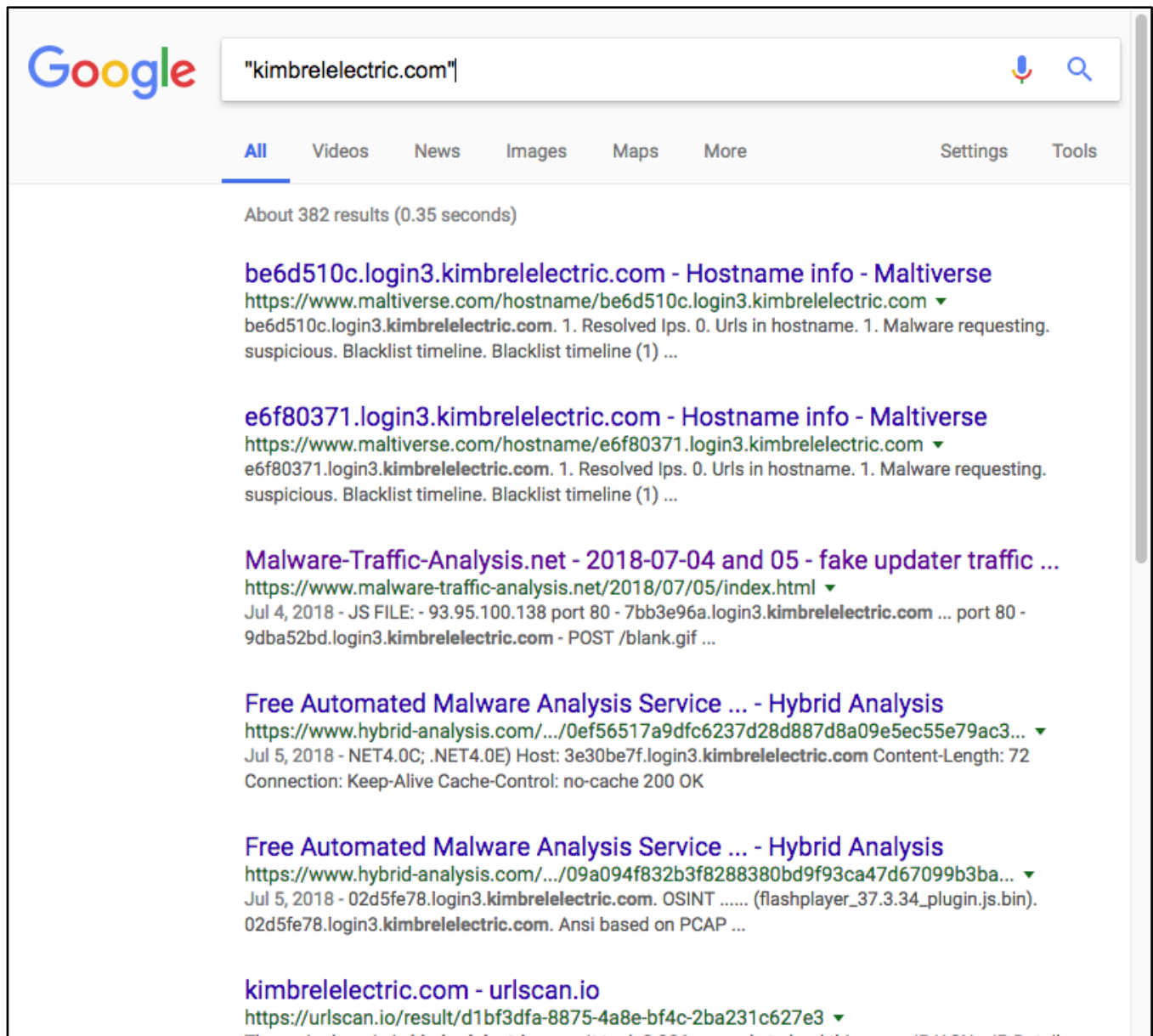


http.request and ip.addr eq 93.95.100.138					
Time	Dst	port	Host	Info	
2018-06-30 20:28...	93.95.100.138	80	bb17eb22.login3.kimbrelelectric.com	POST /blank.gif	
2018-06-30 20:28...	93.95.100.138	80	bb17eb22.login3.kimbrelelectric.com	POST /blank.gif	
2018-06-30 20:28...	93.95.100.138	80	bb17eb22.login3.kimbrelelectric.com	POST /blank.gif	

Shown above: filtering on the HTTP requests to 93.96.100.138.

If you search on the TLD (top-level domain) of "kimbrelelectric.com" on Google, you'll find some interesting results. Remember to use the quotes when you search on domain names, or you might go directly to the domain from your Google search.

2018-06-30 TRAFFIC ANALYSIS EXERCISE: ANSWERS



Shown above: Some of the results for searching "kimbrelectric.com" on Google.

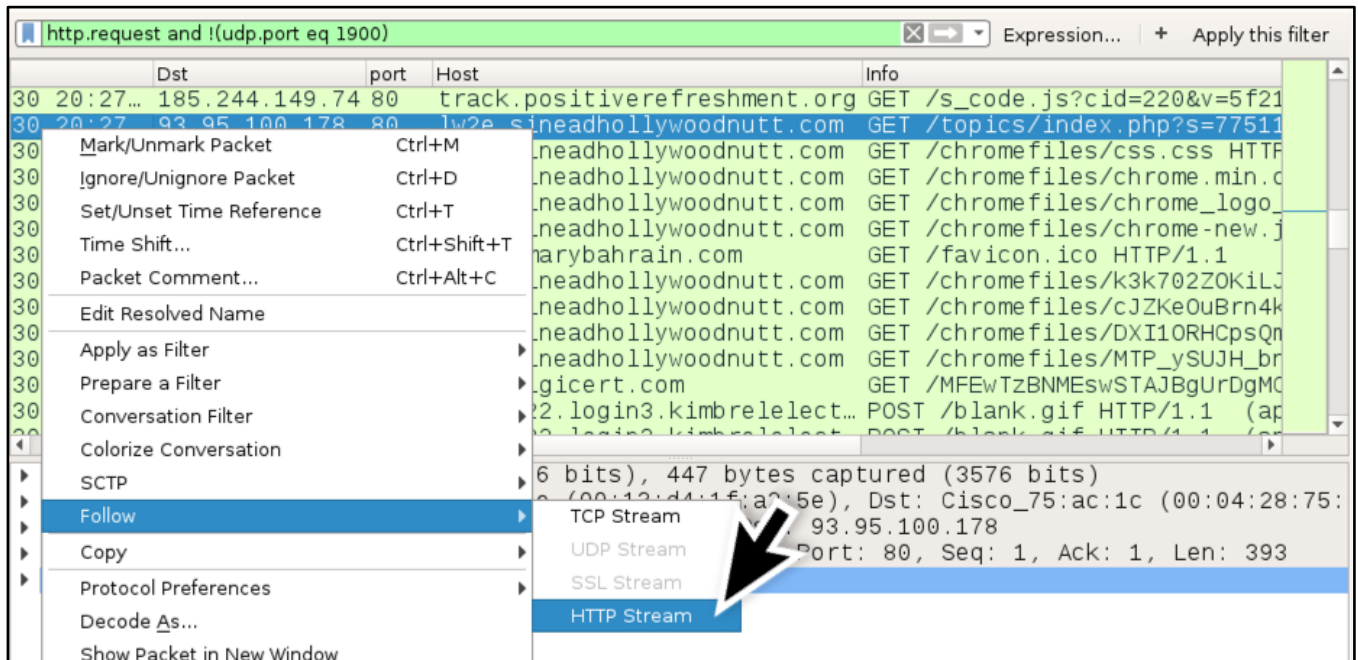
You'll find a [blog post I did on traffic from 2018-07-04 and 05 for fake updates](#), and you'll see some Hybrid-Analysis.com sandbox results for .js files named [flashplayer 37.3.34 plugin.js](#) and [flashplayer 37.3.35 plugin.js](#). Reviewing these articles should reveal this is probably a fake Flash updated page, and my blog post indicates there are different types of payloads delivered by these fake updates.

The alert for **JS.SocGohlish** represents a JS-based social engineering campaigns dubbed "Gholish" which (at this time) I can't find anything further on.

Another clue? There are several HTTP GET requests to **lw2e.sineadhollywoodnutt.com** with "chromefiles" in the URL. If you follow the HTTP stream on the very first HTTP GET request to **lw2e.sineadhollywoodnutt.com** (which doesn't have "chromefiles" in the URL), you'll find

2018-06-30 TRAFFIC ANALYSIS EXERCISE: ANSWERS

indications this is a fake Chrome update page. Real Chrome updates always come from a Google-registered domain name.



Shown above: HTTP GET requests to **lw2e.sineadhollywoodnutt.com** and following HTTP stream for the first HTTP GET request.



Shown above: Title of the HTML page for that first HTTP GET request.

2018-06-30 TRAFFIC ANALYSIS EXERCISE: ANSWERS

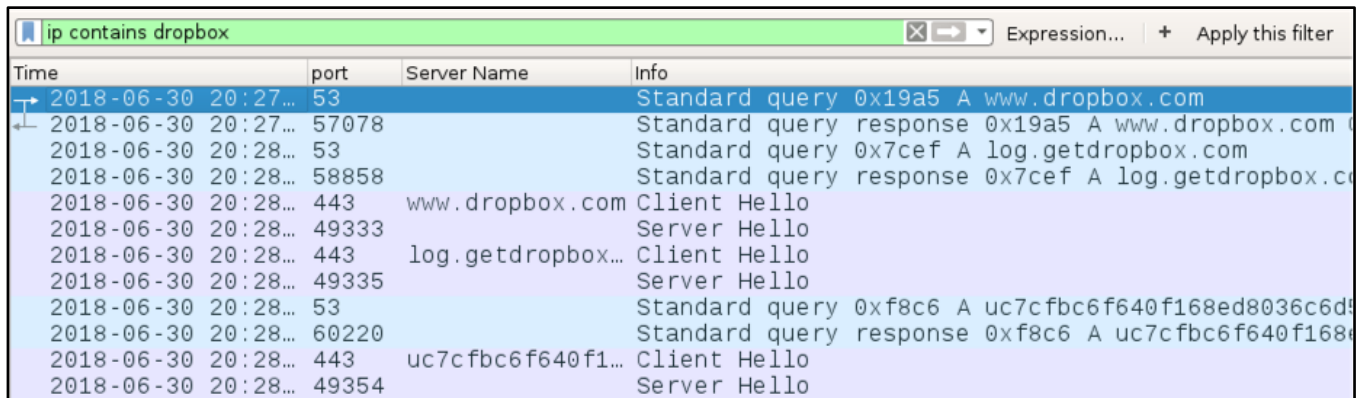


```
nav-l1-more">' +
    <a class="nav-top last-nav-link" href="#">Chromecast <span class="submenu-icon">&nbsp;</span></a>' +
    </li>' +
    </ul>' +
    </div>' +
  </div>' +
  <div class="browser-landing" id="main">' +
    <div class="compact browser-benefits">' +
      <section id="marquee">' +
        <h1 id="main-title" tabindex="-1">You are
using an older version of Chrome</h1>' +
        <p class="browser-promo">Update now to keep
your Chrome browser running smoothly and securely.</p>' +
        <p class="browser-promo">Your download will
begin automatically. If not, click here:</p>' +
        <a class="button edita-download-button
download-button desktop-only hide-cros" href="javascript:void(0)"
id="buttonDownload">Update Chrome</a>' +
        ' +
        </section>' +
      </div>' +
    </div>';
```

2 client pkts, 2 server pkts, 3 turns.

Shown above: Other indicators this is a fake Chrome updater page.

If you review [my blog post based on fake updater traffic from 2017-07-04 and 05](#), you'll find the .js file is delivered through a Dropbox URL. And sure enough, there is HTTPS traffic to dropbox, and it follows the same patterns from my blog post, where I actually have the HTTPS URLs through a Fiddler web proxy capture. Unfortunately, because it's HTTPS traffic, we cannot determine the URL for the .js file from this exercise pcap.



Time	port	Server Name	Info
2018-06-30 20:27...	53		Standard query 0x19a5 A www.dropbox.com
2018-06-30 20:27...	57078		Standard query response 0x19a5 A www.dropbox.com
2018-06-30 20:28...	53		Standard query 0x7cef A log.getdropbox.com
2018-06-30 20:28...	58858		Standard query response 0x7cef A log.getdropbox.com
2018-06-30 20:28...	443	www.dropbox.com	Client Hello
2018-06-30 20:28...	49333		Server Hello
2018-06-30 20:28...	443	log.getdropbox...	Client Hello
2018-06-30 20:28...	49335		Server Hello
2018-06-30 20:28...	53		Standard query 0xf8c6 A uc7cfbc6f640f168ed8036c6d9
2018-06-30 20:28...	60220		Standard query response 0xf8c6 A uc7cfbc6f640f168e
2018-06-30 20:28...	443	uc7cfbc6f640f1...	Client Hello
2018-06-30 20:28...	49354		Server Hello

Shown above: Indicators of HTTPS traffic to dropbox.com.

Based on our research so far, with the fake Chrome update page on **lw2e.sineadhollywoodnutt.com**, HTTPS traffic to **dropbox.com**, and the HTTP POST requests to **blank.gif** on **bb17eb22.login3.kimbrellectric.com**, we can be reasonably sure

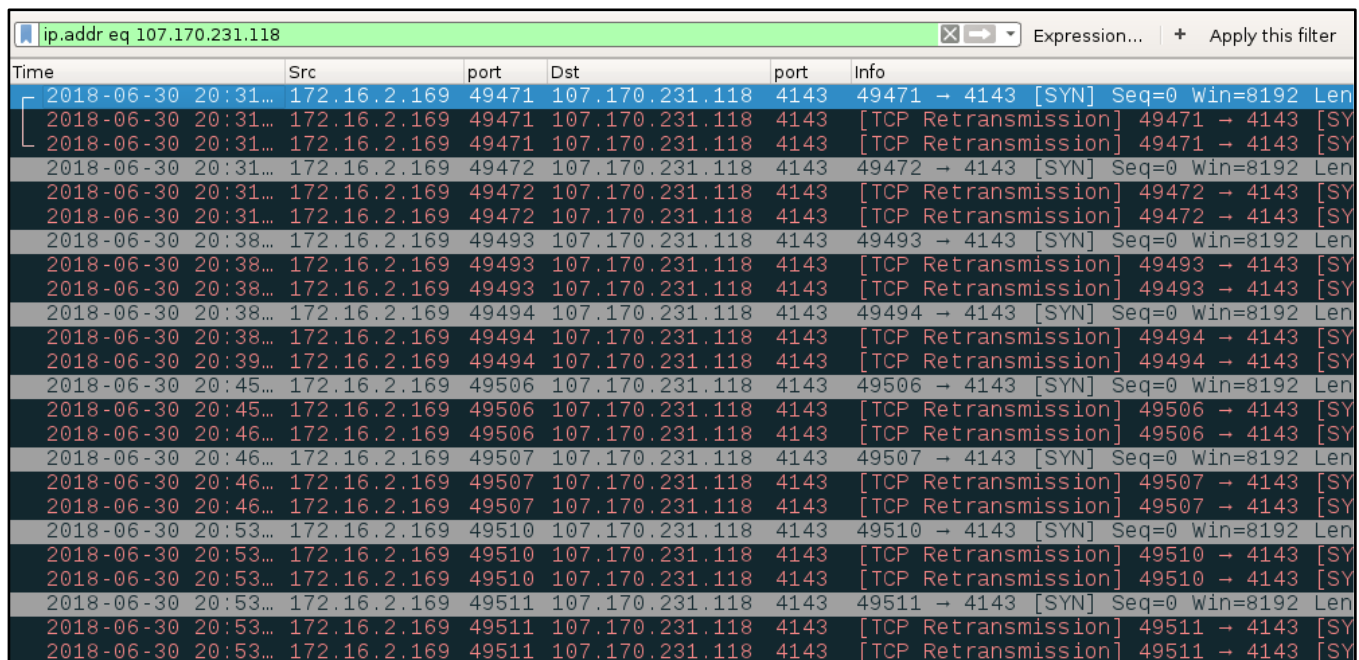
2018-06-30 TRAFFIC ANALYSIS EXERCISE: ANSWERS

the user got a malicious .js file disguised as a fake Chrome update, and that the user clicked on the .js file to run it. But did the user get fully-infected with whatever follow-up malware that .js file would download?

The other alert we haven't looked into yet is:

- 172.16.2.169 49471 - 107.170.231.118 port 80 - **ET CNC Feodo Tracker Reported CnC Server group 2**

As noted on the [Feodo Tracker](#), Version D of Feodo is also known as Dridex, and I've documented that [Dridex is one of the possible payloads from this fake update campaign](#). If you filter the pcap on **ip.addr eq 107.170.231.118**, you'll find attempted TCP connections over TCP port 4143 to 107.170.231.118. The Feodo tracker also shows [TCP port 4143 associated with this IP address as a command and control channel](#).



The screenshot shows a Wireshark packet capture filtered by 'ip.addr eq 107.170.231.118'. The table below represents the data visible in the packet list pane.

Time	Src	port	Dst	port	Info
2018-06-30 20:31...	172.16.2.169	49471	107.170.231.118	4143	49471 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:31...	172.16.2.169	49471	107.170.231.118	4143	[TCP Retransmission] 49471 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:31...	172.16.2.169	49471	107.170.231.118	4143	[TCP Retransmission] 49471 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:31...	172.16.2.169	49472	107.170.231.118	4143	49472 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:31...	172.16.2.169	49472	107.170.231.118	4143	[TCP Retransmission] 49472 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:31...	172.16.2.169	49472	107.170.231.118	4143	[TCP Retransmission] 49472 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:38...	172.16.2.169	49493	107.170.231.118	4143	49493 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:38...	172.16.2.169	49493	107.170.231.118	4143	[TCP Retransmission] 49493 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:38...	172.16.2.169	49493	107.170.231.118	4143	[TCP Retransmission] 49493 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:38...	172.16.2.169	49494	107.170.231.118	4143	49494 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:38...	172.16.2.169	49494	107.170.231.118	4143	[TCP Retransmission] 49494 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:39...	172.16.2.169	49494	107.170.231.118	4143	[TCP Retransmission] 49494 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:45...	172.16.2.169	49506	107.170.231.118	4143	49506 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:45...	172.16.2.169	49506	107.170.231.118	4143	[TCP Retransmission] 49506 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:46...	172.16.2.169	49506	107.170.231.118	4143	[TCP Retransmission] 49506 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:46...	172.16.2.169	49507	107.170.231.118	4143	49507 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:46...	172.16.2.169	49507	107.170.231.118	4143	[TCP Retransmission] 49507 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:46...	172.16.2.169	49507	107.170.231.118	4143	[TCP Retransmission] 49507 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:53...	172.16.2.169	49510	107.170.231.118	4143	49510 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:53...	172.16.2.169	49510	107.170.231.118	4143	[TCP Retransmission] 49510 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:53...	172.16.2.169	49510	107.170.231.118	4143	[TCP Retransmission] 49510 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:53...	172.16.2.169	49511	107.170.231.118	4143	49511 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:53...	172.16.2.169	49511	107.170.231.118	4143	[TCP Retransmission] 49511 → 4143 [SYN] Seq=0 Win=8192 Len=0
2018-06-30 20:53...	172.16.2.169	49511	107.170.231.118	4143	[TCP Retransmission] 49511 → 4143 [SYN] Seq=0 Win=8192 Len=0

Shown above: Attempted TCP connections over TCP port 4143 to 107.170.231.118.

While we're not seeing a full TCP connection and any actual Dridex traffic, that's likely because the server has been taken off-line. An infected Windows host would still try calling out to it. So it's definitely a possibility this Windows host is infected with Dridex. The HTTP POST requests to blank.gif are an indicator the victim double-clicked on a downloaded .js file, and the attempted TCP connections to 107.170.231.118 over port 4143 indicate the user was likely infected with Dridex because of it.

2018-06-30 TRAFFIC ANALYSIS EXERCISE: ANSWERS

The screenshot shows the Feodo Tracker interface. At the top, there's a navigation bar with 'Home', 'Blocklist', and 'Contact'. The main header displays 'Feodo Tracker :: 107.170.231.118'. Below this, the 'C&C Information' section lists details for the IP: Version: D, Host status: offline, Spamhaus SBL: Not listed, AS number: AS14061, AS name: DIGITALOCEAN-ASN - DigitalOcean, LLC, US, Country: US, Firstseen (UTC): 2017-12-16 16:27:12, and Lastseen (UTC): 2018-06-29 09:26:46. The 'Referencing malware binaries' section shows a table of the latest 100 malware binaries referencing this Feodo C&C. The table has columns for Timestamp (UTC), MD5 hash, Filesize, VT, Host, Port, and SSL certificate (SHA1 Fingerprint). The data shows multiple entries for the same host and port (4143) with different MD5 hashes and timestamps. A 'Referencing malware binaries: 7' link is provided below the table. At the bottom, there's a '<< back' link and a copyright notice: 'Copyright © 2018 - feodotracker.abuse.ch'.

Timestamp (UTC)	MD5 hash	Filesize	VT	Host	Port	SSL certificate (SHA1 Fingerprint)
2017-12-16 16:27:12	714577b340e2ee59e49dd70574fa10c8	155'648 bytes	n/a	107.170.231.118	4143	2a21fd57c3aa865ac9cfa6fe6a0843d0072aa18b
2017-12-16 16:27:12	714577b340e2ee59e49dd70574fa10c8	155'648 bytes	n/a	107.170.231.118	4143	8990759bcb9ed72ccb016074e89f8cfd768f62a
2018-02-25 23:18:10	6164228ed2cc0ecea9ce1828d87d827	155'648 bytes	33/68 (48.53%)	107.170.231.118	4143	61c6d4d5ef7aef36275897d6635bb7a58a3e5b11
2018-02-25 23:18:10	6164228ed2cc0ecea9ce1828d87d827	155'648 bytes	33/68 (48.53%)	107.170.231.118	4143	aa6a2c1d61be293408a1bdaaf3da51da49c542e1
2018-02-25 23:18:10	6164228ed2cc0ecea9ce1828d87d827	155'648 bytes	33/68 (48.53%)	107.170.231.118	4143	aaaa549e6e6cd5233a228b4cbb7206a759daf4cb
2018-03-24 13:07:19	042685ffa3dec7a125c5d5945d339724	139'264 bytes	23/62 (37.10%)	107.170.231.118	4143	9366446e08ae06b84ac168a6ef142bb947d208a5
2018-03-24 13:07:19	042685ffa3dec7a125c5d5945d339724	139'264 bytes	23/62 (37.10%)	107.170.231.118	4143	93d565aca5fa4c428b4c7fa1e80b7e7ff70494b3

Shown above: Feodo tracker shows the same TCP port for 107.170.231.118 as we find in our pcap.

OTHER NOTES

The following is other traffic people might think is malicious, but is actually legitimate:

All of those HTTP requests to **gvt1.com** domains at the end of the pcap represent Google Chrome updating itself. They are not malicious and are not related to the fake updater traffic earlier in the pcap.

The screenshot shows the Whois Record for gvt1.com. The 'Domain Profile' section is expanded, showing the 'Registrant Org' as 'Google Inc.', which is circled in red. Below this, the 'Name Servers' are listed: NS1.GOOGLE.COM (has 13,217 domains), NS2.GOOGLE.COM (has 13,217 domains), NS3.GOOGLE.COM (has 13,217 domains), and NS4.GOOGLE.COM (has 13,217 domains).

Shown above: Whois record shows **gvt1.com** registered by Google.

2018-06-30 TRAFFIC ANALYSIS EXERCISE: ANSWERS

The HTTP requests over UDP port 1900 are [SSDP traffic](#), which can discover Plug & Play devices in Microsoft Windows.

Some of the weird DNS queries that also bleed into the NBNS queries are [legitimate traffic caused by Google Chrome](#).

Time	Dst	port	Info
2018-06-30 20:27...	172.16.2.9	53	Standard query 0x18fd A dvdvxgqflqbqg.fabulousdukes.com
2018-06-30 20:27...	172.16.2.9	53	Standard query 0x9244 A iujhiwcwr.fabulousdukes.com
2018-06-30 20:27...	172.16.2.169	56746	Standard query response 0xf90f A rbrsrryqsbctesq.fabulousdukes.co
2018-06-30 20:27...	172.16.2.169	57845	Standard query response 0x18fd No such name A dvdvxgqflq
2018-06-30 20:27...	172.16.2.169	52547	Standard query response 0x9244 No such name A iujhiwcwr.
2018-06-30 20:28...	172.16.2.255	137	Standard query response 0xf90f No such name A rbrsrryqsb
2018-06-30 20:28...	172.16.2.255	137	Name query NB IUJHIWCWR<00>
2018-06-30 20:28...	172.16.2.255	137	Name query NB RBRsRRYQSBCTESQ<00>
2018-06-30 20:28...	172.16.2.255	137	Name query NB DVDVXGQFLQBQG<00>
2018-06-30 20:28...	172.16.2.255	137	Name query NB RBRsRRYQSBCTESQ<00>
2018-06-30 20:28...	172.16.2.255	137	Name query NB DVDVXGQFLQBQG<00>
2018-06-30 20:28...	172.16.2.255	137	Name query NB IUJHIWCWR<00>
2018-06-30 20:28...	172.16.2.255	137	Name query NB IUJHIWCWR<00>
2018-06-30 20:28...	172.16.2.255	137	Name query NB DVDVXGQFLQBQG<00>
2018-06-30 20:28...	172.16.2.255	137	Name query NB RBRsRRYQSBCTESQ<00>

Shown above: Legitimate DNS traffic and NBNS queries from the exercise pcap caused by Google Chrome.

There are probably other items of legitimate traffic you might find weird in this pcap--too many to review here--so I'll just leave it at that for now.